Narration Studio memory vs mem0

How Narration Studio uses "memory" for agent context, compared with mem0.

> **Confirmed** means the behavior is in this repository's source. **Inferred** means it comes from Microsoft Agent Framework (MAF) or from mem0's published design.

Does the app remember conversation context?

Yes — but only in **agent modes** (`handoff` / `sequential`), and it is **not** conversational memory like mem0. Direct mode does not use it.

What the app actually remembers is **pipeline state** (which files exist, what step is next), plus two MAF context providers that inject that state into agent prompts. It does **not** extract facts, embed them, or retrieve relevant memories by similarity.

How this app uses memory

Two providers are attached to every specialist in `AgentFactory` :

```
var memoryRoot = Path.Combine(_store.OutputDirectory, "agent-file-memory");
var fileMemory = new FileMemoryProvider(
    new FileSystemAgentFileStore(memoryRoot),
    _ => new FileMemoryState { WorkingFolder = "production-notes" });

IList<AIContextProvider> providers = [_context, fileMemory];
```

Source: `src/NarrationStudio/Agents/AgentFactory.cs` .

1. ProductionContextProvider (this repo)

File: `src/NarrationStudio/Memory/ProductionContextProvider.cs` .

Read (before an agent call): `ProvideAIContextAsync` injects a live status block into the agent instructions — transcript path, rewritten narration, script/slide/audio/video counts, next step, and naming rules. The status text comes from `PipelineStore.SummarizeForAgent()` .

Write (after an agent call): `StoreAIContextAsync` takes the assistant's reply, truncates it to 240 characters, and appends it to `PipelineState.Notes` (max 24 entries). Tools also append notes such as "Created 8 narration scripts…".

Those notes are saved in `output/pipeline-state.json` . They are **not** fed back into `ProvideAIContextAsync` or `SummarizeForAgent()` . So they are an audit log, not a retrieval loop.

2. FileMemoryProvider (Microsoft Agent Framework)

MAF's file store under `output/agent-file-memory/production-notes`. **Inferred:** it persists agent working notes as files and can re-inject them on later turns in the same process.

Each run **deletes** that folder in `PipelineStore.Initialize`, so it does not survive a rerun.

3. What actually drives the pipeline

`PipelineStore` is the real memory: structured `PipelineState` (segments, paths, status). Agents see it via `GetPipelineStatus` and the context provider. Direct mode uses only this; no chat history, no file memory.

Within one `handoff` run, MAF also keeps the workflow's message stream. That is short-term session context, not long-term memory. **Inferred** from `WorkflowHost.RunAsync`, not implemented in this repo.

How mem0 works

Mem0 is a **memory layer** between the app and the model:

1. After a useful interaction, call `add` to store what should be remembered.
2. Before the next model call, call `search` and put the best hits into the prompt.

Internally, `add` extracts durable facts with an LLM, deduplicates, embeds them, and stores entities. `search` ranks memories against the query (vector similarity, keyword matching, entity overlap), scoped by identifiers such as `user_id`, `agent_id`, or `run_id`.

Storage is typically:

Store	Holds
SQL database	Facts, metadata, history
Vector database	Embeddings for semantic search
Entity / graph store	People, places, concepts and links

Typical loop: search memories → put hits in the system prompt → model answers → `add` the turn.

Comparison

	Narration Studio	mem0
Purpose	Remind agents of **this run's factory status**	Remember **people, preferences, facts** across chats
What is stored	File paths, counts, status, truncated replies	Distilled facts + embeddings + entities
How it is retrieved	Always inject the full status snapshot	Semantic `search(query, user_id)`

Identity	None (one local process)	`user_id`, `agent_id`, `run_id`
Across sessions	Wiped on every run	Survives; that's the point
Storage	JSON + files on disk	SQL + vector DB + entity/graph store
LLM used for memory	No (except the agent itself)	Yes, to extract/update facts
Direct mode	Unused	N/A

This app's loop is: snapshot `PipelineStore` → put the whole status in instructions → model calls tools → tools mutate `PipelineStore`.

It uses MAF **context providers**, not a memory product. It remembers "scripts are 8/8, next is TTS," not "Edwin prefers voice UPXQ…" from last week.

To get mem0-style behavior you would add mem0 (or similar) beside MAF and call `search` / `add` around agent turns. Nothing in this repo does that today.

Key files

File	Role
`src/NarrationStudio/Memory/ProductionContextProvider.cs`	Inject pipeline status; append truncated agent text to `Notes`
`src/NarrationStudio/Agents/AgentFactory.cs`	Wires `ProductionContextProvider` + `FileMemoryProvider` onto every agent
`src/NarrationStudio/Services/PipelineStore.cs`	Structured run state; clears `agent-file-memory` on start
`src/NarrationStudio/Models/PipelineModels.cs`	`PipelineState.Notes`
`src/NarrationStudio/Agents/WorkflowHost.cs`	In-process workflow message stream (short-term, inferred)
`output/pipeline-state.json`	Persisted state + notes (deleted on next run)
`output/agent-file-memory/`	MAF file memory (deleted on next run)